

eIDAS-Node Demo Tools Installation and Configuration Guide v3.0.0

eID Internal

Table of Contents

1	Introduction	4
1.1	Purpose	4
1.2	Document structure	4
1.3	Other technical reference documentation	5
2	Demo Products overview	6
2.1	Integration package	6
2.2	Modules	6
3	Setup configuration directories.....	9
4	Setting up the Demo Service Provider.....	10
5	Setting up the Demo Identity Provider.....	11
6	Setting up the Demo MS-Specific Connector.....	12
7	Setting up the Demo MS-Specific Proxy Service.....	13
8	Additional attributes	15
9	Preparing the installation	16
10	Building and deploying the software	17
10.1	Server deployment.....	17
10.2	Monolithic Deployment.....	18
11	Verifying the installation.....	19
11.1	Configuration files	19
12	Simple protocol	20
12.1	Original SAML EIDAS Request information items	20
12.2	SimpleRequest example	20
12.3	Original SAML EIDAS Response information items	22
12.4	SimpleResponse example	23
13	Demo Tools Migration	27
13.1	Summary of EID Issues	27
13.2	Changes.....	27
13.2.1	Merge wildfly and weblogic profile behavior to default	27
13.2.2	Cleanup of dead code in eidasNode	27
13.2.3	Remove NoJCache modules in favor of jar builds	28
13.2.4	Analyze and Remove Obsolete Configuration Files for JAXB and Binding Context.....	28
13.2.5	Upgrade to Jakarta JEE10	28
13.2.6	Cleanup authors from legacy code.....	30
13.2.7	Re-introduce Hazelcast JCache 5.x.....	30
13.2.8	Remove legacy session-related code from eIDAS-Node modules.....	30

Document history

Version	Date	Modification reason	Modified by
1.0	06/10/2017	Origination	DIGIT
2.0	11/04/2018	Rewritten for version 2.0 to take account of architectural changes with Demo Specific Connector and Demo Specific Proxy Service as well as Demo-SP, Demo IdP.	DIGIT
2.1	09/07/2018	Reuse of document policy updated and version changed to match the corresponding Release. Minor document clarifications made.	DIGIT
2.2	14/09/2018	Minor document clarifications made.	DIGIT
2.3	20/06/2019	Updates related to Jcache Ignite implementation. Other topics that are necessary to migrate.	DIGIT
2.4	06/12/2019	Minor document clarifications made.	DIGIT
2.5	11/12/2020	eIDAS-Node 2.5 release	DIGIT
2.6	15/04/2022	eIDAS-Node 2.6 release	DIGIT
2.7	01/09/2023	eIDAS-Node 2.7 release	DIGIT
2.8	21/05/2024	eIDAS-Node 2.8 release	DIGIT
2.9	04/02/2025	eIDAS-Node 2.9 release	DIGIT
3.0.0	10/10/2025	eIDAS-Node 3.0.0 release	DIGIT

Disclaimer

This document is for informational purposes only and the Commission cannot be held responsible for any use which may be made of the information contained therein. References to legal acts or documentation of the European Union (EU) cannot be perceived as amending legislation in force or other EU documentation.

The document contains a brief overview of technical nature and is not supplementing or amending terms and conditions of any procurement procedure; therefore, no compensation claim can be based on the contents of the present document.

© European Union, 2023

Reuse of this document is authorised provided the source is acknowledged. The Commission's reuse policy is implemented by Commission Decision 2011/833/EU of 12 December 2011 on the reuse of Commission documents.

1 Introduction

This document is intended for a technical audience consisting of developers, administrators and those requiring detailed technical information on how to configure, build and deploy the eIDAS-Node application.

The document describes the installation and configuration settings for the Demo Tools (SP and IdP) supplied with the package for basic testing.

1.1 Purpose

The purpose of this document is to describe how to quickly install the Demo tools provided in the Integration Package (Service Provider (SP), Identity Provider (IdP), Specific Connector and Specific Proxy Service) for testing purposes.

Please note that this is not a guide for your national infrastructure, for implementation options please read the *eIDAS-Node National IdP and SP Integration Guide*.

1.2 Document structure

This document is divided into the following sections:

- Chapter 1 – *Introduction*: this section.
- Chapter 2 – *Demo Products overview* provides information on the deliverable including the package, the modules and dependencies.
- Chapter 3 – *Setup configuration directories* describes the setup configuration directories and environment variables.
- Chapter 4 – *Setting up the Demo Service Provider* provides information on the Demo SP properties to enable set up.
- Chapter 5 – *Setting up the Demo Identity Provider* provides information on the Demo IdP properties to enable set up.
- Chapter 6 – *Setting up the Demo MS-Specific Connector* provides information on the Demo MS-Specific Connector properties to enable set up.
- Chapter 7 – *Setting up the Demo MS-Specific Proxy Service* provides information on the Demo MS-Specific Proxy Service properties to enable set up.
- Chapter 8 – *Additional attributes* describes how to add attributes.
- Chapter 9 – *Preparing the installation* for this information you should refer to the eIDAS-Node Installation and Configuration Guide.
- Chapter 10 – *Building and deploying the software* describes the steps to build and then to deploy the software on the supported servers.
- Chapter 11 – *Verifying the installation* shows the final structure of your application server relevant directories.
- Chapter 12 – *Simple protocol* describes the implementation of Simple Protocol for communication between SP and Specific Connector, and Specific Proxy Service and IdP
- Chapter 13 – *Demo Tools Migration* provides a resume of the topics to be aware in the migration to 2.5 version from previous 2.6.

1.3 Other technical reference documentation

We recommend that you also familiarise yourself with the following eID technical reference documents which are available on **Digital Home > eID**

- *eIDAS-Node Installation, Configuration and Integration Quick Start Guide* describes how to quickly install a Service Provider, eIDAS-Node Connector, eIDAS-Node Proxy Service and IdP from the distributions in the release package. The distributions provide preconfigured eIDAS-Node modules for running on each of the supported application servers.
- *eIDAS-Node Installation and Configuration Guide* describes the steps involved when implementing a Basic Setup and goes on to provide detailed information required for customisation and deployment.
- *eIDAS-Node National IdP and SP Integration Guide* provides guidance by recommending one way in which eID can be integrated into your national eID infrastructure.
- *eIDAS-Node and SAML* describes the W3C recommendations and how SAML XML encryption is implemented and integrated in eID. Encryption of the sensitive data carried in SAML 2.0 Requests and Assertions is discussed alongside the use of AEAD algorithms as essential building blocks.
- *eIDAS-Node Error and Event Logging* provides information on the eID implementation of error and event logging as a building block for generating an audit trail of activity on the eIDAS Network. It describes the files that are generated, the file format, the components that are monitored and the events that are recorded.
- *eIDAS-Node Security Considerations* describes the security considerations that should be taken into account when implementing and operating your eIDAS-Node scheme.
- *eIDAS-Node Error Codes* contains tables showing the error codes that could be generated by components along with a description of the error, specific behaviour and, where relevant, possible operator actions to remedy the error.

Disclaimer: The users of the eIDAS-Node sample implementation remain fully responsible for its integration with back-end systems (Service Providers and Identity Providers), testing, deployment and operation. The support and maintenance of the sample implementation, as well as any other auxiliary services, are provided by the European Commission according to the terms defined in the European Union Public Licence (EUPL) at https://joinup.ec.europa.eu/sites/default/files/custom-page/attachment/eupl_v1.2_en.pdf

2 Demo Products overview

This section provides information on the deliverable including the integration package, the modules and dependencies.

2.1 Integration package

The demo products deliverable consists of the following files:

- SP.war
- IdP.war
- SpecificConnector.war
- SpecificProxyService.war

These are web applications that can be deployed in most available Java web containers.

2.2 Modules

The software is composed of several modules. This section describes the binaries and source code to be installed plus the configuration files.

Table 1: List of modules

Module Name	Folder	Description
Parent	EIDAS-Parent	Module containing a consolidated and consistent location of the libraries and their version number to be used across the different modules.
Light Commons	EIDAS-Light-Commons	Light Common application component and utility classes used for implementing as basis for the EIDAS-Commons and MS Specific Connector and MS Specific Proxy Service modules.
Simple Protocol	EIDAS-SimpleProtocol	Simple Protocol implementation to demonstrate a MS-Specific protocol between SP and Specific-Connector and between IdP and Specific Proxy Service. Not to be used in production.
Commons	EIDAS-Commons	Common Applications components and utility classes for implementing functionality of authentication service.
JCache-Dev	EIDAS-JCache-Dev	Common code for non-distributed JCache implementations
JCache-Dev-Specific-Communication	EIDAS-JCache-Dev-Specific-Communication	Adapts the implementation of non-distributed maps to JCache used in eIDAS-Node MS specific communication caches.
JCache-Ignite	EIDAS-JCache-Ignite	Common code for of Ignite JCache implementations

Module Name	Folder	Description
JCache-Ignite-Specific-Communication	EIDAS-JCache-Ignite-Specific-Communication	Implementation of Ignite JCache for the eIDAS-Node MS specific communication caches
Jcache-Hazelcast	EIDAS-JCache-Hzlcast	Common code for Hazelcast Jcache implementations
Jcache-Hazelcast-Specific-Communication	EIDAS-JCache-Hzlcast-Specific-Communication	Implementation of Hazelcast Jcache for the eIDAS-Node MS specific communication caches
Specific Communication Definition	EIDAS-SpecificCommunicationDefinition	The exchange definition (interfaces) and implementation used to formalise the exchange definition between the Node and the Specific module.
MS Specific Protocol	EIDAS-SimpleProtocol	Module that provides the code to create simple protocol request and response used between the SP and Specific Connector and between IdP and Specific Proxy. Please see appendix for further details. Not to be used in production
MS Specific Connector	EIDAS-SpecificConnector	Demo implementation of Member State (MS) specific connector module. Not to be used in production.
MS Specific Proxy Service	EIDAS-SpecificProxyService	Demo implementation of Member State (MS) specific Proxy Service module. Not to be used in production
Updater	EIDAS-UPDATER	Module used to change configuration of a running eIDAS-Node in testing environment. Not to be used in production
Service provider	EIDAS-SP	Demo implementation of Service Provider module. Not to be used in production
Identity provider	EIDAS-IdP-1.0	Sample of Identity Provider module. Not to be used in production
Basic Setup configuration	EIDAS-Config	Sample configuration as in 12.2.

The figure below shows the dependencies between the installed modules. Note that the modules shown in red are labelled 'DO NOT USE' in the legend, this means use only as samples for demonstration purposes to show that the eIDAS-Node is working, do not use in production. Furthermore, several security vulnerabilities exist and deploying 'as is' in production carries significant risks.

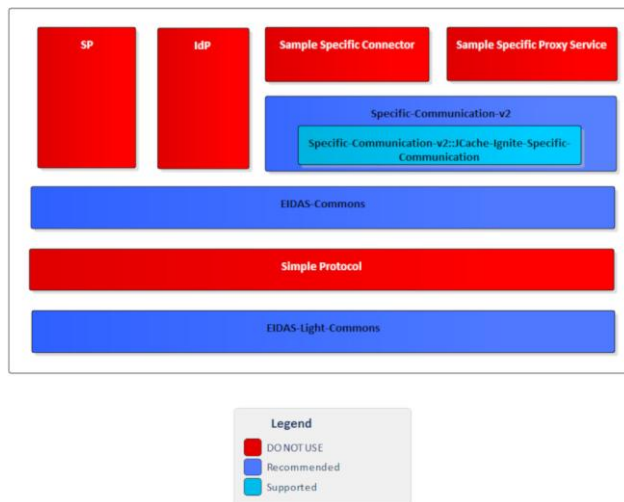


Figure 1: Dependencies between the installed modules

3 Setup configuration directories

This section describes the setup configuration directories and environment variables.

There are five different environment variables used to locate the Demo Tools (Demo-SP, Demo IdP, Demo Specific Connector and Demo Specific Proxy Service) directories of configuration files. These can be defined as OS environment variables or setting it to the runtime environment (by -D switch to JVM or on the AS admin console).

Table 2: Setup configuration directories

Environment variable	Used in	Example target configuration directory
\$SP_CONFIG_REPOSITORY	spEnvironmentContext.xml	file:/C:/PGM/projects/configEidas/sp/
\$SPECIFIC_CONNECTOR_CONFIG_REPOSITORY	specificConnectorApplicationContext.xml	file:/C:/PGM/projects/configEidas/specificConnector/
\$SPECIFIC_PROXY_SERVICE_CONFIG_REPOSITORY	specificProxyServiceEnvironmentContext.xml	file:/C:/PGM/projects/configEidas/specificProxyService/
\$IDP_CONFIG_REPOSITORY	idpEnvironmentContext.xml	file:/C:/PGM/projects/configEidas/idp/

By default, OS environment variables or JVM command line arguments (-D option) must be set in order to specify the location of configuration files.

It is possible to change or hardcode these variables in the following files:

- spEnvironmentContext.xml
- specificConnectorEnvironmentContext.xml
- specificProxyServiceEnvironmentContext.xml
- idpEnvironmentContext.xml
- environmentContext.xml

Please look inside these files to see how it is done.

4 Setting up the Demo Service Provider

This section provides information on the Demo SP properties to enable set up.

The Demo Service Provider (SP) can be used to simulate an MS SP requesting authentication. It works with the default MS-Specific-Connector part using the simple protocol language.

The Basic Setup provides a preconfigured version of Demo Service Provider, however you may need to fine-tune some options.

The Service Provider *sp.properties* configuration details are described in the following table. The location of this file must be set by the SP_CONFIG_REPOSITORY environment variable or command line argument.

Table 3: Service Provider Properties

Key	Description
provider.name	Provider Name for this Service Provider
sp.return	URL used when the eIDAS-Node Connector finishes the process. This must be the value of the machine running the Service Provider, its format is <i>http://sp.ip.address:sp.port.number/sp.deployment.name/ReturnPage</i> .

The following table describes the available eIDAS-Node for this Service Provider.

Table 4: Available eIDAS-Node for Service Provider

Key	Description
country.number	The number of possible eIDAS-Nodes that can communicate with this SP
countryX.name	The name of the eIDAS-Node X(= positive integer)
countryX.url	The URL for the eIDAS-Node X. This must be the value of the machine running the eIDAS-Node using the format: <i>http://node.ip.address:node.port.number/node.deployment.name/</i> This URL is used by the SP to send its request.

5 Setting up the Demo Identity Provider

This section provides information on the Demo IdP properties to enable set up.

The Demo Identity Provider (IdP) can be used to simulate an MS IdP requesting authentication. It works with the default MS-Specific-Proxy-Service part using the simple protocol language.

In order to proceed with the Basic Setup, you may need to modify the configuration of the Demo Identity Provider.

The `user.properties` holds the credentials for citizens who are able to log in. The format is: `<username>=<password>`.

The `idp.properties` is used by the IdP to provide the attribute values in the format: `<username>.<attributeName>=<attributeValue>`.

Table 5: Sample of user.properties content

Key	Description
myUser=myPassword	A sample username and password
myUser.LegalName=my legal name	A sample attribute definition

The `idp.properties` holds configuration parameters about the application. The location of this file must be set by the `IDP_CONFIG_REPOSITORY` environment variable or command line argument.

Table 6: Identity Provider Properties

Key	Description
idp.issuer	Issuer name for the IdP.

6 Setting up the Demo MS-Specific Connector

This section provides information on the Demo MS-Specific Connector properties to enable set up.

The eIDAS-Node integration package contains a Demo Member State Specific Connector part that is aligned with the use of Demo SP.

There are some configuration items that might need to be customised according to the test environment. The configuration file name is *specificConnector.xml*, and is located by SPECIFIC_CONNECTOR_CONFIG_REPOSITORY environment variable or command line argument.

Table 7: Specific Connector part properties

Key	Description
issuer.name	Name of the issuer. Responses sent will have this value as issuer.
distributedMapsSpecificConnector	Boolean value (true false), which indicates if the application will activate distributed maps feature, necessary if clusters are used.
specific.connector.request.url	The URL of the Node to send the binary light token related to the Light Request.
relaystate.randomize.null	Boolean value (true false), to activate or de-activate the behaviour of populating a null relayState with a random value.

7 Setting up the Demo MS-Specific Proxy Service

This section provides information on the Demo MS-Specific Proxy Service properties to enable set up.

The eIDAS-Node integration package contains a Demo Member State Specific Proxy Service part that is aligned with the use of Demo IdP.

There are some configuration items that might need to be customised according to the test environment. The configuration file name is *specificProxyService.xml*, and is located by SPECIFIC_PROXY_SERVICE_CONFIG_REPOSITORY environment variable or command line argument.

Table 8: Specific part properties

Key	Description
issuer.name	Name of the issuer for the IdP. Responses sent will have this value as issuer.
distributedMapsSpecificProxyService	Boolean value (true false), which indicates if the application will activate distributed maps feature, to be used in cluster mode.
idp.url	URL to where the MS request will be sent.
specific.proxyservice.idp.response.service.url	URL to where the MS Specific Proxy Service can receive the response from the Demo IdP. It is sent in the request to the IdP.
ask.consent.request	Boolean value (true false), which indicates if the application will activate the consent pages for the request. If set to "true", the Consent Page will be displayed to the user when processing the request from the eIDAS-Node Connector. Attributes without consent will be removed from the response.
ask.consent.response	Boolean value (true false), which indicates if the application will activate the consent pages for the response. If set to "true", the Value Consent Page (CV) will be displayed before sending the response to the eIDAS-Node Connector. The user is able to cancel the forwarding of authentication data, resulting in an authentication failure.
ask.consent.response.show.only.eidas.attributes	Boolean value (true false), which indicates if the application will activate the display of the response's attribute names. Depends on the activation of ask.consent.response If set to "true" only the Core eIDAS attributes/values will be displayed. On "false", the Value Consent Page (CV) will display all the Response

Key	Description
	attributes/values, including additional (specified in XML file) ones.
ask.consent.response.show.attribute.values	Boolean value (true false), which indicates if the application will activate the display of the response's attribute values. Depends on the activation of ask.consent.response If set to "true", the Value Consent Page (CV) will display attribute names and values for the Response, "false" will result in attribute names only.
consent.Request.LightToken.Secret	Secret to be used in the request consent.
consent.Request.LightToken.Algorithm	Digest Algorithm for the request consent
consent.Response.LightToken.Secret	Secret to be used in the response consent
consent.Request.LightToken.Algorithm	Digest Algorithm for the response consent
default.specific.proxyservice.idp.response.service.url	URL where the MS Specific Proxy Service can receive the response from the Demo IdP. It is sent in the request to the IdP when specific modules are included in the Node as JAR.
specific.proxyservice.response.url	The URL of the Node to send the binary light token related to the Light Response.
relaystate.randomize.null	Boolean value (true false), to activate or de-activate the behaviour of populating a null relayState with a random value.

8 Additional attributes

This section describes how to add attributes.

To add additional attributes, use the files named *additional-attributes.xml*, located in the environment variables:

- \$SPECIFIC_CONNECTOR_CONFIG_REPOSITORY
- \$SPECIFIC_PROXY_SERVICE_CONFIG_REPOSITORY

or by command line argument. The file *eidas-attributes.xml* should remain unchanged.

The following table contains the additional attribute keys that need to be present to add an additional attribute.

Table 9: Additional attributes

Key	Description
1.NameUri	URI of the attribute.
1.FriendlyName	Friendly name of the attribute.
1.PersonType	PersonType, either natural or legal, corresponding to the Natural and Legal Persons
1.Required	If the attribute is to be set as required.
1.XmlType.NamespaceUri	The additional attribute namespace URI.
1.XmlType.LocalPart	The additional attribute local part.
1.XmlType.NamespacePrefix	The additional attribute's namespace prefix.
1.AttributeValueMarshaller	The additional attribute's namespace value marshaller.

To add a second attribute, you will need to increment the prefix number (i.e. the additional attribute would be prefixed "2" and so on).

Also, the same has to be done in the eIDAS-Node configuration file for these additional attributes to be recognised.

9 Preparing the installation

For instructions on how to prepare the servers: Tomcat or WildFly before deploying the Demo Tools please refer to the *eIDAS-Node Installation and Configuration Guide*.

10 Building and deploying the software

This section describes the steps to build and then to deploy the software on the supported servers.

The project build files are in **Maven3** format, so you need to install Maven. Download instructions are provided at <http://maven.apache.org/run-maven/index.html>). Recommended versions of Maven are 3.3.9 and above. Lower versions can result in exceptions.

There are two ways to build the binaries from sources:

- **Parent build:** the *pom.xml* file in the EIDAS-Parent module is a common reference for all dependent module/external Maven artefact versions, and able to build all binaries related to EidasNode and/or Demo Tools

There are various profiles to help tailor the build to one's particular needs. These profiles relate to the scope of modules to be built, specifically NodeOnly (this is active by default) and DemoToolsOnly.

For instance, issuing the Maven "install" command with the appropriate activation profile (e.g., -P NodeOnly,DemoToolsOnly) will result in a full build.

- **Module-based build:** it is possible to build the artifacts one by one, which can be helpful if there is a need to build just one module. In this case please don't forget the dependencies between them. There is a certain order that needs to be followed.

The next sections detail the above two methods for supported application servers.

10.1 Server deployment

You must compile, install and deploy the projects, either by compiling the parent project (Table 10: Parent project build for an application server) or by compiling each module separately in the order shown below (Table 11: Module-based build for an application server).

At a command prompt, first navigate to the folder indicated and then enter the corresponding command line.

In order to deploy the project, after the build is complete, copy the artifacts needed IdP.war, SP.war, SpecificConnector.war and SpecificProxyService.war to the deploy folder of the Server. The EidasNodeConnector.war and EidasNodeProxy.war may also be needed, check eIDAS-Node Installation and configuration guide for more information.

Application server	Deploy folder
Tomcat 11.0.4	\$TOMCAT_HOME/webapps/
WildFly 35.0.1.Final	\$WILDFLY_HOME/standalone/deployments/

Table 10: Parent project build for an application server

Step	Folder	Command line
1	Project root folder	<pre>mvn clean install --file EIDAS-Parent/pom.xml -P [NodeOnly,]DemoToolsOnly -P specificCommunicationJcacheIgnite specificCommunicationJcacheHazelcast specificCommunicationJcacheDev [-DspecificJar]</pre>

Table 11: Module-based build for an application server

Step	Folder	Command line
1	EIDAS-Parent	mvn clean install -N
2	EIDAS-Light-Commons	mvn clean install
3	EIDAS-Commons	mvn clean install
4	EIDAS-JCache-Dev	mvn clean install
5	EIDAS-JCache-Dev-Specific-Communication	mvn clean install
6	EIDAS-JCache-Ignite	mvn clean install
7	EIDAS-JCache-Ignite-Specific-Communication	mvn clean install
8	EIDAS-Jcache-HzIcast	mvn clean install
9	EIDAS-Jcache-HzIcast-Specific-Communication	mvn clean install
10	EIDAS-SpecificCommunicationDefinition	mvn clean install -P specificCommunicationJcacheIgnite specificCommunicationJcacheHazelcast specificCommunicationJcacheDev [-DspecificJar]
11	EIDAS-SimpleProtocol	mvn clean install
12	EIDAS-SpecificConnector	mvn clean install
13	EIDAS-SpecificProxyService	mvn clean install
14	EIDAS-SP	mvn clean package
15	EIDAS-IdP-1.0	mvn clean package

10.2 Monolithic Deployment

Besides the 'Basic Deployment' described in this document, a 'Monolithic Deployment' is possible. In this case the *EidasNodeConnector.war* will include the *SpecificConnector* module and the *EidasNodeProxy.war* will include the *SpecificProxyService* module as JAR.

In this case add *-D specificJar* to the build commands for the following modules:

- EIDAS-SpecificCommunicationDefinition
- EIDAS-SpecificConnector
- EIDAS-SpecificProxyService
- EIDAS-SP
- EIDAS-IdP-1.0

This also applies to EidasNode modules, so please check the *Monolithic Deployment* section in the *eIDAS-Node Installation and Configuration Guide* for more details.

11 Verifying the installation

This section shows the final structure of your application server relevant directories; so that you can confirm that you have made the proper configurations. The structure of the application's 'war' files is also shown so you can verify that your applications were built successfully.

```
EidasNodeConnector.war
EidasNodeProxy.war
IdP.war
SpecificConnector.war
SpecificProxyService.war
SP.war
(server specific directories were not included)
```

Code block 1 Binaries in \$TOMCAT_HOME/webapps/ or \$WILFDLY_HOME/standalone/Deployments/

11.1 Configuration files

The following configuration and keystore files are needed for the full installation with Demo Tools. The layout itself can be different, depending on the environment variables, so this is just an example of Basic Setup:

- server/idp/additional-attributes.xml
- server/idp/eidas-attributes.xml
- server/idp/idp.properties
- server/idp/user.properties
- server/sp/additional-attributes.xml
- server/sp/eidas-attributes.xml
- server/sp/sp.properties
- server/specificConnector/ignite/igniteSpecificCommunication.xml (needed only if profile specificCommunicationJcacheIgnite is activated)
- server/specificConnector/hazelcast/hazelcastSpecificCommunication.xml (needed only if profile specificCommunicationJcacheHazelcast is activated)
- server/specificConnector/additional-attributes.xml
- server/specificConnector/eidas-attributes.xml
- server/specificConnector/specificCommunicationDefinition.xml
- server/specificConnector/specificConnector.xml
- server/specificProxyService/ignite/igniteSpecificCommunication.xml (needed only if profile specificCommunicationJcacheIgnite is activated)
- server/specificProxyService/hazelcast/hazelcastSpecificCommunication.xml (needed only if profile specificCommunicationJcacheHazelcast is activated)
- server/specificProxyService/additional-attributes.xml
- server/specificProxyService/eidas-attributes.xml
- server/specificProxyService/specificCommunicationDefinition.xml
- server/specificProxyService/specificProxyService.xml

12 Simple protocol

Simple Protocol has been implemented for communication between SP and Specific Connector, and Specific Proxy Service and IdP. The main goal is to show the concept of integrating SPs, IdPs or similar entities with an eIDAS-Node. This is a simplified protocol for demonstration purposes only. It does not include security features.

The Simple Protocol was not designed to be used 'as is' by Member States, only for demonstration purposes. Some parts of it may evolve/be changed in future versions.

12.1 Original SAML EIDAS Request information items

Request

```

AuthnRequest
  ID
  Destination
  ForceAuthn
  IssueInstant
  ProviderName
  Version
  AssertionConsumerServiceURL
  SPType
  RequestedAuthnContext
    Comparison
    AuthnContextClassRef
  RequestedAttributes
    RequestedAttribute
      FriendlyName
      isRequired
      Value
        LatinScript
      Value

```

12.2 SimpleRequest example

SimpleRequest

```

{
  "authentication_request": {
    "attribute_list": [
      {
        "type": "requested_attribute",
        "name": "D-2012-17-EUIdentifier",
        "required": false
      },
      {
        "type": "requested_attribute",
        "name": "EORI",
        "required": false
      },
      {
        "type": "requested_attribute",
        "name": "LEI",
        "required": false
      },
      {
        "type": "requested_attribute",
        "name": "LegalName",
        "required": true
      }
    ],
    "requested_authentication_context": {
      "comparison": "minimum",
      "context_class": [
        "A"
      ],
      "non_notified_context_class": [
        "http://non.eidas.eu/NotNotified/LoA/1"
      ]
    },
    "citizen_country": "CA",
    "created_on": "2020-06-12T18:11:25.107+02:00",
    "force_authentication": true,
    "id": "835a09f7-0179-4198-9c19-36680d0fe531",
    "name_id_policy": "unspecified",
    "provider_name": "DEMO-SP-CA",
    "requester_id": "http://eidas.eu/EidasNode/RequesterId",
    "serviceUrl": "http://localhost:8080/SP/ReturnPage",
    "sp_type": "public",
    "version": "1"
  }
}

```

Note: If an attribute value is supplied in the Request, that will be a valueattribute, so "type" will change from "requested_attribute" to a certain type.

Simple Protocol	LightRequest	Mandatory Yes/No	Nature
authentication_request	lightRequest	No	abstract
version		Yes	always "1"
id	id to map	Yes	UUID generated
created_on		Yes	timestamp, local time in json "de facto" format

Simple Protocol	LightRequest	Mandatory Yes/No	Nature	
force_authentication		No	always "true"	
provider_name	providerName	No	string	
serviceUrl		Yes	URL for the Response	
sp_type	spType	No	"public" "private" omitted	
requester_id	requesterId	No	string	
context_class	levelOfAssurance type="notified"	Yes	Context_class	LevelOfAssurance
non_notified_context_class	levelOfAssurance type="nonNotified"		"A" "B"	" http://eidass.europa.eu/LoA/low "
			"C" "D"	" http://eidass.europa.eu/LoA/substantial "
			"E"	" http://eidass.europa.eu/LoA/high "
			+ Optionally non notified LoA	
citizen_country	citizenCountryCode	Yes	This was an HTTP parameter with SAML, now it is the part of the message body. Value: ISO Country Code e.g. "CA"	
name_id_policy	nameIDPolicy	No	Can be omitted OR any of these values: "persistent" "transient" "unspecified" To map: persistent => urn:oasis:names:tc:SAML:2.0:nameid-format:persistent transient => urn:oasis:names:tc:SAML:2.0:nameid-format:transient unspecified => urn:oasis:names:tc:SAML:1.1:nameid-format:unspecified	
attribute_list	immutableAttributeMap (please check example above)	No	Abstract, the idea is to use the FriendlyName attribute of eIDAS attributes here, then the AttributeRegistry.getByFriendlyName can be used in the mapping. It is possible to add a prefix such as "sp_"	

Attribute type is always 'requested_attribute' for Request.

12.3 Original SAML EIDAS Response information items

Response

```

Response
  Destination
  ID
  InResponseTo
  IssueInstant
  Version
  Issuer
  Status
    StatusCode
      StatusCode
    StatusMessage
  Assertion
    Issuer
    Subject
      NameID
      NameQualifier
      Value
      SubjectConfirmation
        Method
        SubjectConfirmationData
          Address
          InResponseTo
          NotOnOrAfter
          Recipient
      Conditions
        NotBefore
        NotOnOrAfter
  AudienceRestriction
    Audience
  AuthnStatement
    AuthnInstant
    AuthnContext
      AuthnContextClassRef
      AuthnContextDecl
  AttributeStatement
    Attribute
      FriendlyName
      Name
      NameFormat
      AttributeValue
        LatinScript

```

12.4 SimpleResponse example

SimpleResponse

Success:

```

{
  "response": {
    "version": "1",
    "id": "0a88c46e-24a7-4194-90f1-35485977bb18",
    "destination" : "http://", <----- TO BE DECOMISSIONED, NO
EIDINT yet
    "inresponse_to": "e7d5db08-0818-449f-bec2-d257bf9593d7",
    "created_on": "2012-04-23T20:28:43.511+02:00",
    "authentication_context_class": "high",
    "client_ip_address": "123.0.0.2",
    "issuer": "DEMO-IDP",
    "subject": "ES/BE/0123456",
    "name_id_format": "transient",
    "status": {
      "status_code": "success",
    },
    "attribute_list": [
      {
        "type": "string",
        "name": "gender",
        "value": "Male"
      },
      {
        "type": "string_list",
        "name": "birth_name",
        "values": [
          {
            "latin_script": false,
            "value": "Árvíztűrő Tükörfúrógép"
          },
          {
            "value": "Arvizturo Tukorfurogep"
          }
        ]
      },
      {
        "type": "date",
        "name": "date_of_birth",
        "value": "1905-04-20"
      },
      {
        "type": "address",
        "name": "current_address",
        "value": {
          "po_box": "1234",
          "locator_designator": "28",
          "locator_name": "DIGIT building",
          "cv_address_area": "Etterbeek",
          "thoroughfare": "Rue Belliard",
          "post_name": "ETTERBEEK CHASSE",
          "admin_unit_first_line": "BE",
          "admin_unit_second_line": "ETTERBEEK",
          "post_code": "1040"
        }
      }
    ]
  }
}

```

Error:

```

{
  "response" : {
    "version" : "1",
    "id" : "0a88c46e-24a7-4194-90f1-35485977bb18",
    "inresponse_to" : "e7d5db08-0818-449f-bec2-d257bf9593d7",
    "created_on" : "2012-04-23T20:28:43.511+02:00",
    "issuer" : "DEMO-IDP",
    "status" : {
      "status_code" : "failure",
      "sub_status_code" : "AuthnFailed",
      "status_message" : "all hands on deck"
    }
  }
}

```

Simple Protocol	LightResponse	Mandatory Yes/No	Nature
response	LightResponse	No	abstract
version		Yes	always "1"
id	ID to map	Yes	UUID generated
inresponse_to	Original req ID to map	Yes	Mandatory
subject	Subject	No	New field for the <i>user.properties</i> (eg.: xavi.subject) Only if message is SUCCESS!
name_id_format	NameIDFormat	No	At the IDP, copy the value of NameIDPolicy from the Request Only if message is SUCCESS!
client_ip_addresses	IPAddress	No	optional address of the client browser
created_on		Yes	timestamp, local time in json "de facto" format
authentication_context_class	LevelOfAssurance	No	"high" "substantial" "low" Or non-notified LoA values
issuer	Issuer	Yes	string
status	Status	Yes	abstract structure
status_code	StatusCode	No	mandatory, allowed values: success failure To be mapped as full SAML2Core URN (see SAML2Core): success => "urn:oasis:names:tc:SAML:2.0:status:Success" responder failure => "urn:oasis:names:tc:SAML:2.0:status:Responder" responder failure => "urn:oasis:names:tc:SAML:2.0:status:Requester" (not covered: "urn:oasis:names:tc:SAML:2.0:status:VersionMismatch" because it is for the Proxy Node in our simple implementation)

Simple Protocol	LightResponse	Mandatory Yes/No	Nature
sub_status_code	SubStatusCode	No	To be mapped as SAML:Core secondary status code like AuthnFailed, attach this string to the URN (see SAML2Core), optional: only in case of failure. Possible values: AuthnFailed InvalidAttrNameOrValue InvalidNameIDPolicy NoAuthnContext NoAvailableIDP NoPassive NoSupportedIDP PartialLogout PartialLogout PartialLogout ProxyCountExceeded RequestDenied RequestUnsupported RequestVersionDeprecated RequestVersionTooHigh RequestVersionTooLow ResourceNotRecognized TooManyResponses UnknownAttrProfile UnknownPrincipal UnsupportedBinding The strategy here is just to append "urn:oasis:names:tc:SAML:2.0:status:" in Specific Proxy, and remove it in the Specific Connector. The IDP should implement some of these (as appropriate) but not all e.g.: AuthnFailed should be the failure case when the credentials entered in the IDP are wrong.
status_message	StatusMessage	No	Only in case of failure. IDP should be able to produce some example text (e.g. "failed to authenticate because of bad credentials" for the "AuthnFailed" code)
attribute_list	ImmutableAttributeMap (please check example above)	No	Abstract, the idea is to use the FriendlyName attribute of EIDAS attributes here, then the AttributeRegistry.getByFriendlyName can be used in the mapping. It is possible to add a prefix such as "idp_". Only if message is SUCCESS!

Possible attribute types are: *string*, *string_list*, *date* and *address*. Add JAXB implementing class if more required.

13 Demo Tools Migration

In this section it is briefly described, the relevant changes in the Demo Tools worth mentioning occurred from previous version related either to code or configuration.

13.1 Summary of EID Issues

EID-XYZ

13.2 Changes

13.2.1 Merge wildfly and weblogic profile behavior to default

The profiles for wildfly and weblogic in the pom.xml files were removed, and their configurations were moved to the build section. These tasks are now handled in the build section, ensuring they are executed automatically for all environments without requiring profile activation.

13.2.1.1 Code changes

- EIDAS-IdP-1.0
 - Deleted the weblogic profile, and its configuration was moved to the build section in the pom.xml
- EIDAS-SP
 - Deleted the wildfly and weblogic profiles and their configurations were moved to the build section in pom.xml
- EIDAS-SpecificConnector
 - Deleted the wildfly and weblogic profiles and their configurations were moved to the build section in pom.xml
- EIDAS-SpecificProxyService
 - Deleted the wildfly and weblogic profiles and their configurations were moved to the build section in pom.xml

13.2.2 Cleanup of dead code in eidasNode

Unused code was removed as part of the cleanup process. The removal focused on maintaining the system's integrity by carefully analyzing the relevance of flagged code. Certain elements were intentionally retained due to potential indirect usage or framework-specific requirements, such as methods or configurations that may be utilized through reflection, serialization, XML binding, or dependency injection.

13.2.2.1 Code changes

The cleanup was conducted across the following modules:

- EIDAS-IdP-1.0
- EIDAS-SP
- EIDAS-SpecificConnector
- EIDAS-SpecificProxyService

13.2.3 Remove NoJCache modules in favor of jar builds

The NoJcache modules have been removed in favour of new build profiles, which accomplish the same functionality.

13.2.3.1 Code changes

- EIDAS-SpecificConnector
 - Added NoJcache build profile
- EIDAS-SpecificProxyService
 - Added NoJcache build profile
- Removed EIDAS-SpecificConnector-NoJcache module
- Removed EIDAS-SpecificProxyService-NoJcache module

13.2.4 Analyze and Remove Obsolete Configuration Files for JAXB and Binding Context

Removed the files `com.sun.xml.ws.spi.db.BindingContextFactory` and `javax.xml.bind.JAXBContext` as they were no longer necessary

13.2.4.1 Code changes

- EIDAS-SpecificConnector
 - Removed `com.sun.xml.ws.spi.db.BindingContextFactory` file because it was used in older WebLogic versions for JAXB RI configuration, but it is no longer required in the current environment.
 - Removed `javax.xml.bind.JAXBContext` file because it referenced the outdated `javax` namespace, while the project now uses `jakarta`, making the file obsolete.
- EIDAS-SpecificProxyService
 - Removed `com.sun.xml.ws.spi.db.BindingContextFactory` file because it was used in older WebLogic versions for JAXB RI configuration, but it is no longer required in the current environment.
 - Removed `javax.xml.bind.JAXBContext` file because it referenced the outdated `javax` namespace, while the project now uses `jakarta`, making the file obsolete.

13.2.5 Upgrade to Jakarta JEE10

The `eidasNode` was upgraded from Java EE to Jakarta EE 10. This required updating various dependencies, renaming some version properties, adjusting dependency scopes, and fixing issues to align with the Jakarta EE 10 specifications.

13.2.5.1 Code changes

- EIDAS-IdP-1.0
 - several java classes were updated to replace `javax` imports with `jakarta`
 - replaced imports for the Struts library to align with the updated package structure
 - replaced `setServletRequest` and `setServletResponse` with `withServletRequest` and `withServletResponse` to align with the updated Struts API in `ProcessLogin.java`
 - updated the Struts DTD reference from `http://struts.apache.org/dtds/struts-2.5.dtd` to `http://struts.apache.org/dtds/struts-6.5.dtd` and added `<constant name="struts.parameters.requireAnnotations.transitionMode" value="true"/>` to

enable a smooth transition to the updated Struts parameter requirements in struts.xml

- updated the <web-app> declaration from Java EE 2.4 to Jakarta EE 6.0 in web.xml to align with the Jakarta EE namespace and schema requirements
- replaced the OWASP Java Encoder tag library URI with owasp.encoder.jakarta in callback.jsp
- replaced Java EE dependencies with Jakarta EE equivalents, including javax.servlet.jsp-api with jakarta.servlet.jsp-api, javax.json with jakarta.json, and added jakarta.xml.bind-api, while removing outdated exclusions and dependencies, ensuring compatibility with Jakarta EE 10 in pom.xml
- EIDAS-SP
 - several java classes were updated to replace javax imports with jakarta
 - replaced imports for the Struts library to align with the updated package structure
 - replaced setServletRequest and setServletResponse with withServletRequest and withServletResponse to align with updated Struts API, added attrJsonMapForDisplay and convertToFriendlyNameMap to enhance attribute mapping for UI display in ReturnAction.java
 - updated the Struts DTD reference from http://struts.apache.org/dtds/struts-2.5.dtd to http://struts.apache.org/dtds/struts-6.5.dtd and added <constant name="struts.parameters.requireAnnotations.transitionMode" value="true"/> to enable a smooth transition to the updated Struts parameter requirements in struts.xml
 - updated the <web-app> declaration from Java EE 2.4 to Jakarta EE 6.0 in web.xml to align with the Jakarta EE namespace and schema requirements
 - refactored the s:iterator logic to directly iterate over attrJsonMap entries instead of extracting keys first, simplifying the structure and improving readability in returnPage.jsp
 - updated pom.xml by replacing javax.json with jakarta.json, replacing javax.servlet.jsp-api with jakarta.servlet.jsp-api, adding jakarta.xml.bind-api, and ensuring all dependencies align with Jakarta EE to maintain compatibility with Jakarta-based frameworks
- EIDAS-SpecificConnector
 - several java classes were updated to replace javax imports with jakarta
 - updated web.xml by replacing the java.sun.com namespace with jakarta.ee and updating the JSTL context parameter from javax.servlet.jsp.jstl.fmt.locale to jakarta.servlet.jsp.jstl.fmt.locale
 - replaced the OWASP Java Encoder tag library URI with owasp.encoder.jakarta in several jsps
 - updated pom.xml by replacing javax.json with jakarta.json, adding jakarta.xml.bind-api and jakarta.el-api, replacing encoder-jsp with encoder-jakarta-jsp, and ensuring all dependencies align with Jakarta EE for improved compatibility with Jakarta-based frameworks
- EIDAS-SpecificProxyService
 - several java classes were updated to replace javax imports with jakarta
 - updated web.xml by replacing the java.sun.com namespace with jakarta.ee and updating the JSTL context parameter from javax.servlet.jsp.jstl.fmt.locale to jakarta.servlet.jsp.jstl.fmt.locale
 - replaced the OWASP Java Encoder tag library URI with owasp.encoder.jakarta in several jsps

- updated pom.xml by replacing javax.json with jakarta.json, adding jakarta.xml.bind-api and jakarta.el-api, replacing encoder-jsp with encoder-jakarta-jsp, and ensuring all dependencies align with Jakarta EE for compatibility with Jakarta-based frameworks

13.2.6 Cleanup authors from legacy code

Removed legacy Javadoc @author and @version tags containing personal information.

13.2.6.1 Code changes

Cleaned up Javadoc tags in multiple files across the following module:

- EIDAS-SP

13.2.7 Re-introduce Hazelcast JCache 5.x

Hazelcast was re-introduced as caching alternative for ignite

13.2.7.1 Code changes

- EIDAS-SpecificConnector
 - added hazelcast to exclusions for 'asDependency'
- EIDAS-SpecificProxy
 - added hazelcast to exclusions for 'asDependency'

13.2.8 Remove legacy session-related code from eIDAS-Node modules

Removed legacy session-related code from several modules. Deleted the SessionHolder class and all its usages from the code.

13.2.8.1 Code changes

- EIDAS-SpecificConnector
 - deleted the SessionHolder class